# Transformers Learn Generalizable CoT Reasoning via Gradient Descent

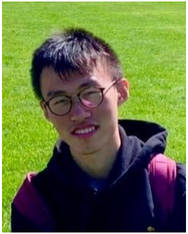
**Yuejie Chi**

CS Colloquium

Yale Statistics & Data Science

December 2025

Tong Yang
CMU



Zixin Wen
CMU

Yu Huang
Penn

Yingbin Liang
OSU

Yuxin Chen
Penn

Aarti Singh
CMU

# Large language models

The language models predict the next word based on previous words.

`Prompt: Explain reinforcement learning (RL).`

`Answer: Reinforcement learning (RL) is a type of machine learning where an...`

**Large language models (LLMs)**

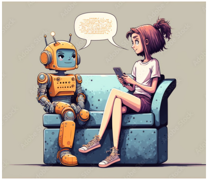
apple

image

...

agent

Applications in translation, Q&A, summarization...



# Large language models

The language models predict the next word based on previous words.

`Prompt: Explain reinforcement learning (RL).`

`Answer: Reinforcement learning (RL) is a type of machine learning where an agent`

**Large language models (LLMs)**

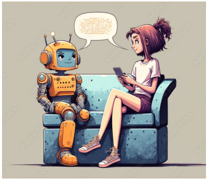
learns

sleeps

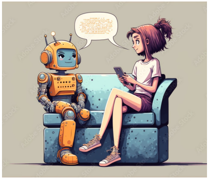
...

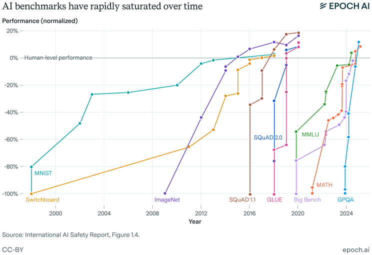
eats

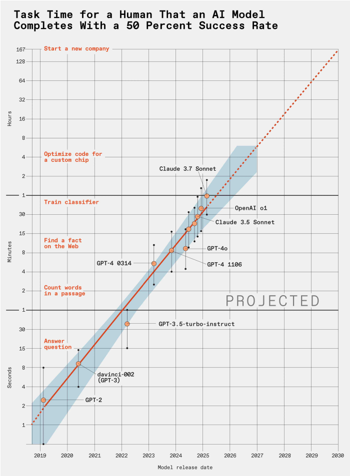
Applications in translation, Q&A, summarization...



# The rise of large language models



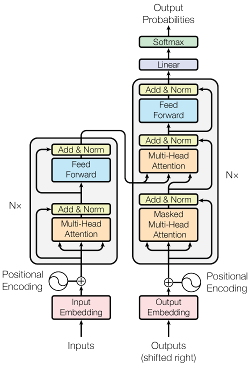



LLM benchmarking shows capabilities doubling every **7 months**.

https://spectrum.ieee.org/large-language-model-performance

# Transformers as the backbone architecture





Attention is all you need

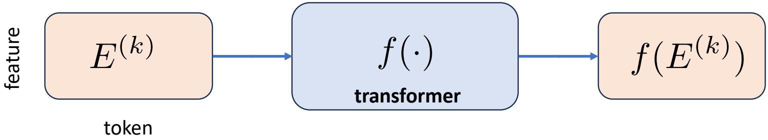
A Vaswani, N Shazeer, N Parmar… - Advances in neural …, 2017 - proceedings.neurips.cc

The dominant sequence transduction models are based on complex recurrent orconvolutional neural networks in an encoder and decoder configuration. The best …

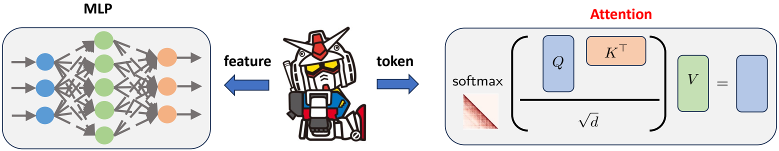
☆ Save  99 Cite  Cited by 188409  Related articles  »

# Transformers as the backbone architecture

**Sequence-to-sequence:**

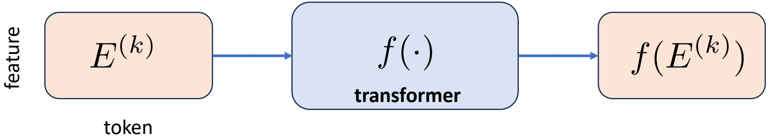



**Transformer block = attention + MLP**

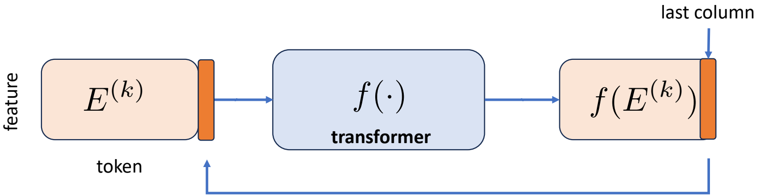


# Transformers as the backbone architecture

**Sequence-to-sequence:**




**Autoregressive decoding:**



# Chain-of-thought (CoT)

LLMs solve harder questions by reasoning **step-by-step**: generate intermediate reasoning steps before inferring an answer.*

**Standard Prompting**

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27.

**Chain-of-Thought Prompting**

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

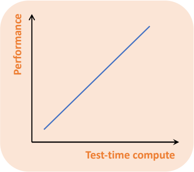
A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9.

<Question→Answer>

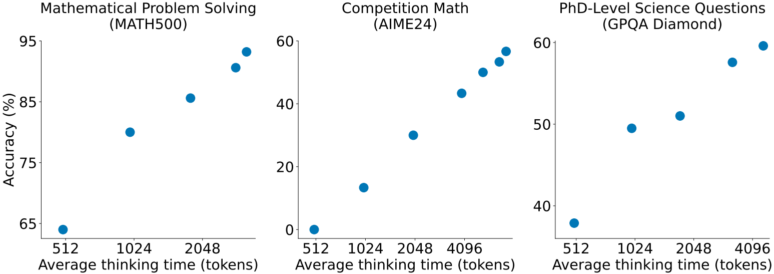
<Question→Rationale→Answer>

*Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.

# Test-time scaling

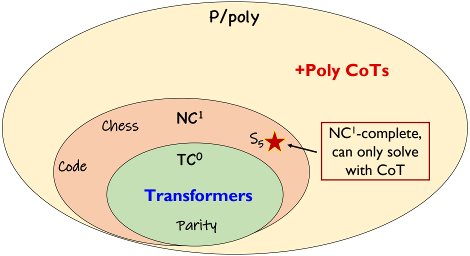



**Reasoning performance improves with more test-time compute (e.g., thinking longer).**




(Figure credit: s1: Simple test-time scaling)

# Expressive power of CoT



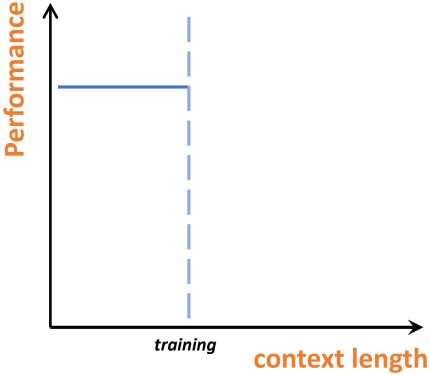

assuming $TC^0 \neq NC^1$

**Expressiveness:** (Li et al., 2024; Merrill et al., 2023; Feng et al., 2023...)

- without CoT $\subset$ $\mathsf{TC}^0$;
- with linear CoTs $\supseteq$ $\mathsf{NC}^1$;
- with polynomial CoTs $\supseteq$ $P$/poly.

With increasing CoT lengths, transformers become more powerful!

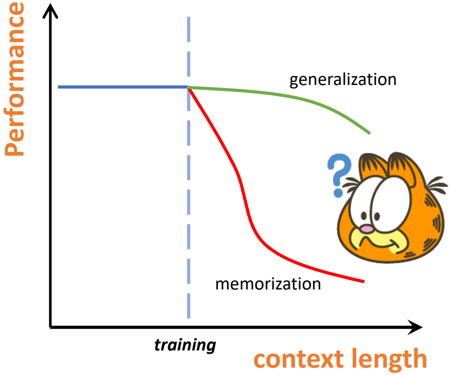
# Length generalization from training perspective




Can transformers trained on shorter tasks be generalized to longer ones?

No optimization guarantees *beyond* $\mathrm{TC}^0$.

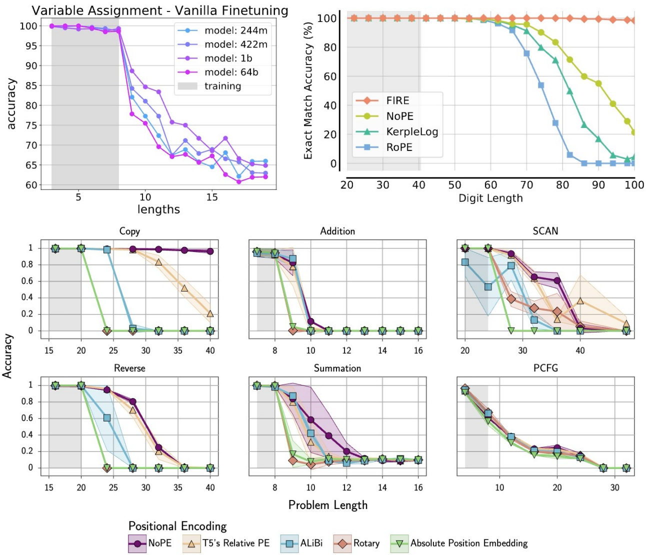
# Length generalization from training perspective




Can transformers trained on shorter tasks be generalized to longer ones?

No optimization guarantees *beyond* $\mathrm{TC}^0$.

# Empirical evidence is mixed




(Figure credit: Anil et al., 2022; Kazemnejad et al., 2023; Zhou et al., 2024)

# This talk: training dynamics of CoT reasoning

Why do *trained* transformers perform multi-step reasoning?

**Optimization**

**Generalization**

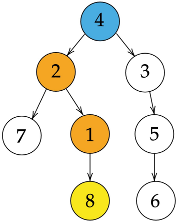
- What does gradient descent converge to?
- What is the trained attention at convergence?

- Will trained transformers generalize to unseen tasks?
- Will trained transformers generalize to longer lengths?

**Take-away:** one-layer transformers with CoT training can solve reasoning tasks that are otherwise out of reach

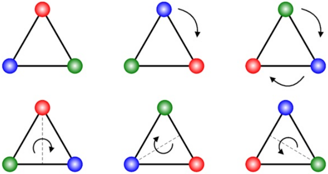
# This talk: two case studies with one-layer transformers

**symbolic reasoning with multi-head attention**




Path-finding in a tree
(Brinkman et al., 2024)

**reasoning with group action with attention + MLP**



LEGO
(Zhang et al., 2023)

# *Multi-head transformers learn symbolic multi-step reversal reasoning*

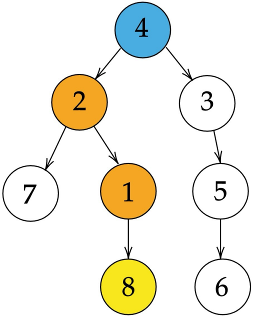
Tong Yang
CMU

Yu Huang
Penn

Yingbin Liang
OSU

"Multi-head Transformers Provably Learn Symbolic Multi-step Reasoning via Gradient Descent,"
T. Yang, Y. Huang, Y. Liang, Y. Chi, *NeurIPS 2025*

# Pathfinding in trees




Prompt:

Edge set: $1 \rightarrow 8,\ 4 \rightarrow 2,\ 2 \rightarrow 7, 3 \rightarrow 5,$
$2 \rightarrow 1,\ 4 \rightarrow 3,\ 5 \rightarrow 6$

Goal: $8$

Root: $4$

Path-finding task:

Goal-to-root (backward):

$$8 \rightarrow 1 \rightarrow 2 \rightarrow 4$$

Root-to-goal (forward):

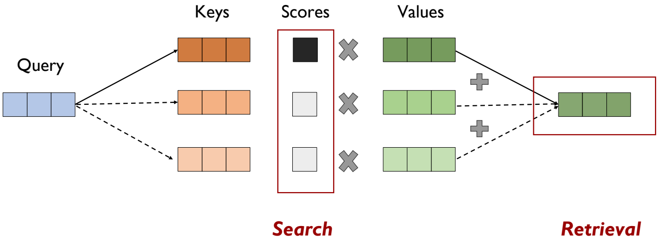
$$4 \rightarrow 2 \rightarrow 1 \rightarrow 8$$

- **Backward** reasoning: find goal-to-root path; easy by backtracking.
- **Forward** reasoning: find the root-to-goal path; harder and needs to first solve backward task and then reverse it.

# Attention head

**Attention head:** For each query $q$, it matches against all the key-value pairs $(k_i, v_i)$ and compute the value $v$:

$$\text{head}(q) = \sum \alpha_i v_i, \quad \text{where} \quad \alpha_i = \frac{\exp(q^\top k_i/\sqrt{d})}{\sum_u \exp(q^\top k_u/\sqrt{d})}.$$

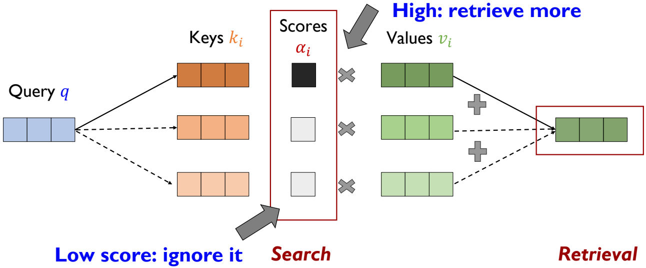



**Attention** searches and retrieves the values of relevant tokens

# Attention head

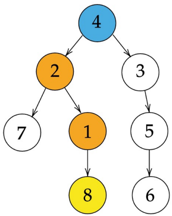
**Attention head:** For each query $q$, it matches against all the key-value pairs $(k_i, v_i)$ and compute the value $v$:

$$\text{head}(q) = \sum \alpha_i v_i, \quad \text{where} \quad \alpha_i = \frac{\exp(q^\top k_i/\sqrt{d})}{\sum_u \exp(q^\top k_u/\sqrt{d})}.$$




**Attention** searches and retrieves the values of relevant tokens

# Attention mechanism for one-step backward reasoning




$$E^{(0)} = \begin{bmatrix} X^{(0)} \\ Y^{(0)} \end{bmatrix} = \left[\begin{array}{ccccccc|c|c} \multicolumn{7}{c|}{\text{edges}} & \text{r} & \text{g} \\ 1 & 4 & 2 & 3 & 2 & 4 & 5 & 0 & 8 \\ 8 & 2 & 7 & 5 & 1 & 3 & 6 & 4 & 0 \end{array}\right] \begin{bmatrix} 1 \\ 8 \end{bmatrix}$$

*query*

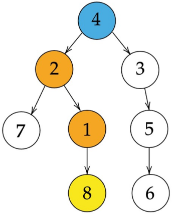
**Attention** searches the edge and retrieves the parent node

**Attention output:** simplified model with trainable $B$, where $x_{-1}^{(k-1)}$ is the last entry of $X^{(k-1)}$:

$$\widehat{o}^{(k)} = \underbrace{\begin{pmatrix} X^{(k-1)} \\ Y^{(k-1)} \end{pmatrix}}_{\text{keys}} \cdot \underbrace{\text{softmax}\left(Y^{(k-1)\top} B x_{-1}^{(k-1)}\right)}_{\text{scores}}.$$

- Attend to $(p(n^{(k-1)}), n^{(k-1)})$, and output $(p(n^{(k-1)}), n^{(k-1)})$.

# Attention mechanism for one-step backward reasoning



$$E^{(0)} = \begin{bmatrix} X^{(0)} \\ Y^{(0)} \end{bmatrix} = \left[\begin{array}{ccccccc|c|c} \multicolumn{7}{c|}{\overbrace{\hphantom{1\ 4\ 2\ 3\ 2\ 4\ 5}}^{\text{edges}}} & \overbrace{\hphantom{0}}^{\text{r}} & \overbrace{\hphantom{8}}^{\text{g}} \\ 1 & 4 & 2 & 3 & 2 & 4 & 5 & 0 & 8 \\ 8 & 2 & 7 & 5 & 1 & 3 & 6 & 4 & 0 \end{array}\right] \begin{bmatrix} 1 \\ 8 \end{bmatrix}$$

***search***

***query***

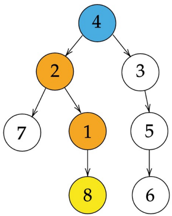
**Attention** searches the edge and retrieves the parent node

**Attention output:** simplified model with trainable $B$, where $x_{-1}^{(k-1)}$ is the last entry of $X^{(k-1)}$:

$$\widehat{o}^{(k)} = \underbrace{\begin{pmatrix} X^{(k-1)} \\ Y^{(k-1)} \end{pmatrix}}_{\text{keys}} \cdot \underbrace{\mathrm{softmax}\left( Y^{(k-1)\top} B x_{-1}^{(k-1)} \right)}_{\text{scores}}.$$

- Attend to $(p(n^{(k-1)}), n^{(k-1)})$, and output $(p(n^{(k-1)}), n^{(k-1)})$.

# Attention mechanism for one-step backward reasoning




$$E^{(0)} = \begin{bmatrix} X^{(0)} \\ Y^{(0)} \end{bmatrix} = \left[ \overbrace{\begin{matrix} 1 & 4 & 2 & 3 & 2 & 4 & 5 \\ 8 & 2 & 7 & 5 & 1 & 3 & 6 \end{matrix}}^{\text{edges}} \;\middle|\; \overbrace{\begin{matrix} 0 \\ 4 \end{matrix}}^{\text{r}} \;\middle|\; \overbrace{\begin{matrix} 8 \\ 0 \end{matrix}}^{\text{g}} \right] \begin{bmatrix} 1 \\ 8 \end{bmatrix}$$

***retrieval*** ***search*** ***query***

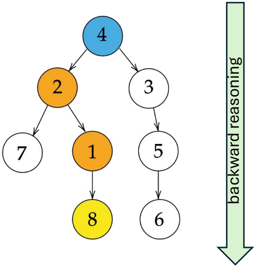
**Attention** searches the edge and retrieves the parent node

**Attention output:** simplified model with trainable $B$, where $x_{-1}^{(k-1)}$ is the last entry of $X^{(k-1)}$:

$$\widehat{o}^{(k)} = \underbrace{\begin{pmatrix} X^{(k-1)} \\ Y^{(k-1)} \end{pmatrix}}_{\text{keys}} \cdot \underbrace{\mathrm{softmax}\left( Y^{(k-1)\top} B x_{-1}^{(k-1)} \right)}_{\text{scores}}.$$

- Attend to $(p(n^{(k-1)}), n^{(k-1)})$, and output $(p(n^{(k-1)}), n^{(k-1)})$.

# Attention construction for backward reasoning




$$\begin{array}{c} \\ \text{edges}\ \ \text{r}\ \ \text{g} \quad \text{output} \end{array}$$

$$E^{(0)} = \begin{bmatrix} X^{(0)} \\ Y^{(0)} \end{bmatrix} = \begin{bmatrix} 1 & 4 & 2 & 3 & 2 & 4 & 5 & 0 & 8 \\ 8 & 2 & 7 & 5 & 1 & 3 & 6 & 4 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 8 \end{bmatrix}$$

$$E^{(1)} = \begin{bmatrix} X^{(1)} \\ Y^{(1)} \end{bmatrix} = \begin{bmatrix} 1 & 4 & 2 & 3 & 2 & 4 & 5 & 0 & 8 & 1 \\ 8 & 2 & 7 & 5 & 1 & 3 & 6 & 4 & 0 & 8 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

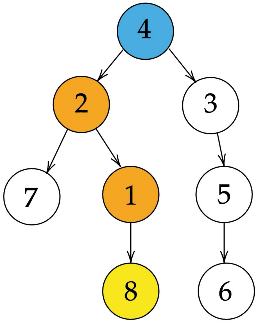
$$E^{(2)} = \begin{bmatrix} X^{(2)} \\ Y^{(2)} \end{bmatrix} = \begin{bmatrix} 1 & 4 & 2 & 3 & 2 & 4 & 5 & 0 & 8 & 1 & 2 \\ 8 & 2 & 7 & 5 & 1 & 3 & 6 & 4 & 0 & 8 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 2 \end{bmatrix}$$

$$E^{(3)} = \begin{bmatrix} X^{(3)} \\ Y^{(3)} \end{bmatrix} = \begin{bmatrix} 1 & 4 & 2 & 3 & 2 & 4 & 5 & 0 & 8 & 1 & 2 & 4 \\ 8 & 2 & 7 & 5 & 1 & 3 & 6 & 4 & 0 & 8 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 4 \end{bmatrix}$$

**Theorem (Construction for backward reasoning)**

*Given linearly independent nodal embeddings, there exist attention constructions that implement this mechanism.*

# Task switching for forward reasoning




Root-to-goal (forward):

$$\underbrace{8 \to 1 \to 2 \to 4}_{\text{Stage 1 backtracking}} \to \underbrace{2 \to 1 \to 8}_{\text{Stage 2 CoT Retrieval}}$$

**Task switch:** To implement forward reasoning, we need to solve and automatically switch between two subtasks:

- Backtracking: find the goal-to-root path and store it in CoT;
- CoT retrieval: retrieve the root-to-goal path from CoT.

# Two-head attention for forward reasoning

**Embedding of the input tree with two heads:**

$$E_{\mathsf{r2g}}(\mathcal{T}) = \begin{pmatrix} X(\mathcal{T}) \\ Y(\mathcal{T}) \\ Z(\mathcal{T}) \end{pmatrix} = \begin{pmatrix} x_1 & \dots & x_{l(\mathcal{T})} & a_0 & a_0 \\ y_1 & \dots & y_{l(\mathcal{T})} & a_{r(\mathcal{T})} & a_{g(\mathcal{T})} \\ s_f & \dots & s_f & s_f & s_b \end{pmatrix},$$

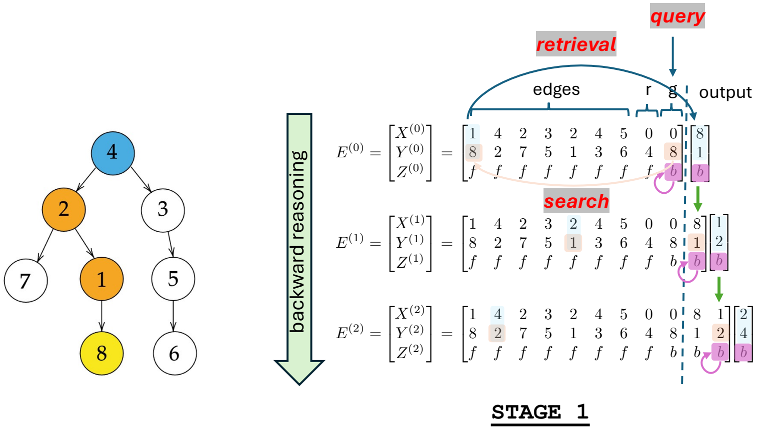
where $s_f, s_b \in \mathbb{R}^{d_2}$ are stage token embeddings.

**Attention output:** simplified model with trainable $\{B_i, C_i\}_{i\in[3]}$:

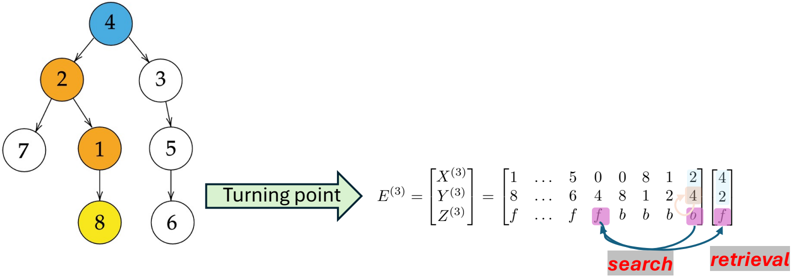
$$\widehat{o}^{(k)} = \begin{pmatrix} \begin{pmatrix} Y^{(k-1)} \\ X^{(k-1)} \end{pmatrix} \cdot \mathsf{softmax}\left(X^{(k-1)\top} B_1 y_{-1}^{(k-1)} + Y^{(k-1)\top} B_2 y_{-1}^{(k-1)} + Z^{(k-1)\top} B_3 z_{-1}^{(k-1)}\right) \\ Z^{(k-1)} \cdot \mathsf{softmax}\left(X^{(k-1)\top} C_1 y_{-1}^{(k-1)} + Y^{(k-1)\top} C_2 y_{-1}^{(k-1)} + Z^{(k-1)\top} C_3 z_{-1}^{(k-1)}\right) \end{pmatrix}.$$

- The first head performs edge retrieval;
- The second head controls the task switch.

# Attention mechanism for forward reasoning: Stage 1

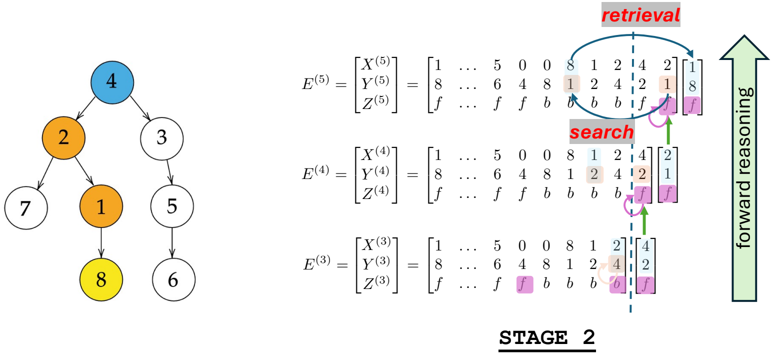



- First head: attend to $(p(n^{(k-1)}), n^{(k-1)})$ (avoid attending to itself via stage signal embedding), and output $(n^{(k-1)}, p(n^{(k-1)}))$.
- Second head: attend to and output (its own) stage signal $s_b$.



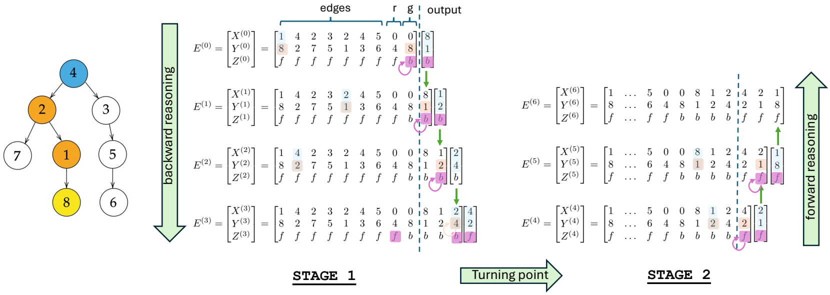

- First head: attend to $(y_{-1}^{(k-1)}, x_{-1}^{(k-1)})$ (itself) via null node embedding at root, and output $(x_{-1}^{(k-1)}, y_{-1}^{(k-1)})$.
- Second head: attend to and output stage signal $s_f$ via null node embedding at root.

# Attention mechanism for forward reasoning: Stage 2




- First head: attend to $(n^{(k-1)}, p(n^{(k-1)}))$ generated in *STAGE 1*, and output $(p(n^{(k-1)}), n^{(k-1)})$.
- Second head: attend to and output (its own) stage signal $s_f$.

# Attention construction for forward reasoning




**Theorem (Construction for forward reasoning)**

*Given linearly independent nodal embeddings, there exist attention constructions that implement this mechanism.*

- Ensures the feature interplays to enable autonomous switching.

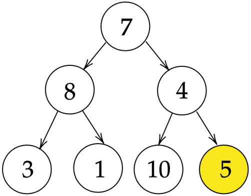
# Assumptions for training dynamics

Can trained transfomers learn these mechanisms?

# Assumptions for training dynamics

Can trained transfomers learn these mechanisms?

**Training distribution:** Let $m \geq 3$ and $S \geq 2^{m+1} - 1 =: N$. The training distribution $P_{\text{train}}$ is the **uniform distribution** over **perfect binary trees** $\mathcal{T} = (\mathcal{V}(\mathcal{T}), \mathcal{E}(\mathcal{T}), g(\mathcal{T}))$ of depth $m$ with distinct nodes chosen from $[S]$ and a goal node $g(\mathcal{T})$ being one of the leaf nodes.




**Orthonormal embeddings:** (i) $\{a_i\}_{i \in [S]}$ are orthonormal vectors in $\mathbb{R}^{d_1}$; (ii) $s_f, s_b$ are orthonormal vectors in $\mathbb{R}^{d_2}$.

# Training objective

**Autoregressive teacher-forcing:** at each step $k \geq 2$, the input

$$E_{\text{train}}^{(k-1)}(\mathcal{T}) = (E_{\text{train}}^{(k-2)}(\mathcal{T}), o^{(k-1)}(\mathcal{T}))$$

is the concatenation of the input at step $k-1$ and the $(k-1)$-th column of the ground truth output. The output is

$$\widehat{o}_{\text{train}}^{(k)}(\mathcal{T};\theta) = f(E_{\text{train}}^{(k-1)}(\mathcal{T});\theta)_{:,-1}.$$

**CoT training loss:**

$$\mathcal{L}_{\text{train}}(\theta) = \frac{1}{2}\mathbb{E}_{\mathcal{T}\sim P_{\text{train}}}\|O(\mathcal{T}) - \widehat{O}_{\text{train}}(\mathcal{T};\theta)\|_{\mathsf{F}}^2.$$

**Gradient descent:** we initialize all parameters to be zero, and

$$\theta^{(t+1)} = \theta^{(t)} - \eta\nabla_\theta\mathcal{L}_{\text{train}}(\theta^{(t)}), \quad \forall t \geq 0.$$

# Theory of optimization

**Theorem (Training dynamics)**

*We use zero initialization. For learning rate $\eta$ and $\epsilon$ that are sufficiently small, we reach*

$$\mathcal{L}_{\mathsf{train}}(\theta^{(t)}) \leq \epsilon$$

*as soon as the number of iterations satisfies*

- *For backward reasoning*

$$T = \mathcal{O}\left(\frac{mS}{\eta\epsilon}\log\left(\frac{Nm}{\epsilon}\right)\right).$$

- *For forward reasoning*

$$T = \mathcal{O}\left(\frac{S}{\eta}\left(\frac{m}{\epsilon}\right)^{3/2}\log\left(\frac{N}{\epsilon}\right)\right).$$

- Backward reasoning converges at a rate of $\widetilde{\mathcal{O}}(1/\epsilon)$, and forward reasoning converges at a rate of $\widetilde{\mathcal{O}}(1/\epsilon^{3/2})$.

# Theory of generalization

**Generalization error:** on unseen tree $\mathcal{T}$

$$\mathcal{L}_{\text{test}}(\mathcal{T};\theta) = \frac{1}{2}\|O(\mathcal{T}) - \widehat{O}(\mathcal{T};\theta)\|_{\mathsf{F}}^2.$$

**Theorem (Generalization)**

*Given any tree $\mathcal{T}$ at test time with $\widetilde{N}$ distinct nodes chosen from $[S]$ and a path length $\widetilde{m}$, there exists small enough $\epsilon_0 = \epsilon_0(m, \widetilde{N}, \widetilde{m}) > 0$ such that for any $\epsilon \in (0, \epsilon_0]$, when the training less $\mathcal{L}_{\text{train}}(\theta^{(t)}) \leq \epsilon$,*

- *For backward reasoning,*

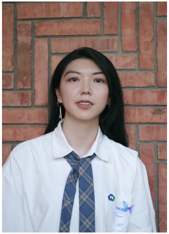
$$\mathcal{L}_{\text{test}}(\mathcal{T};\theta^{(t)}) \leq 4\max\left\{\left(\frac{\widetilde{N}+\widetilde{m}-1}{N+m-2}\right)^2, 1\right\} \cdot \frac{\widetilde{m}}{m}\epsilon.$$

- *For forward reasoning,*

$$\mathcal{L}_{\text{test}}(\mathcal{T};\theta) \leq 4\max\left\{\left(\frac{\widetilde{N}+2\widetilde{m}-1}{N+2m-1}\right)^2, \left(\frac{\widetilde{N}}{N}\right)^2, 1\right\}\epsilon.$$

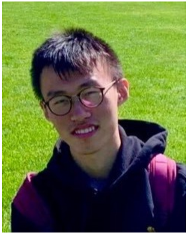
- Better length generalization on unseen trees for small training error.

# *Transformers learn multi-step compositional reasoning with length generalization*

Yu Huang$^\star$

Penn



Zixin Wen$^\star$

CMU

Aarti Singh

CMU

Yuxin Chen

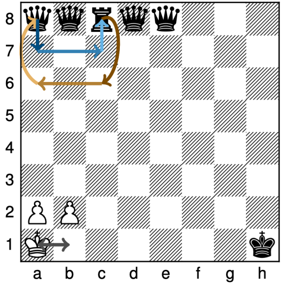
Penn


"Transformers Provably Learn Chain-of-Thought Reasoning with Length Generalization,"
Y. Huang$^*$, Z. Wen$^*$, A. Singh, Y. Chi, Y. Chen, *NeurIPS 2025*. $^*$ = equal contribution.

# State tracking

Given an initial state and a sequence of *actions*, compute the final state.

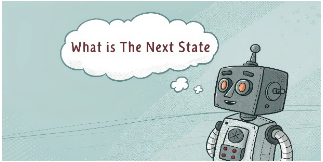
### Playing Chess




Images modified from Merrill et al., 2024

### Code evaluation

```
x = [0, 0, 1, 0, 0]
x[1], x[3] = x[3], x[1] # Swap 1, 3
```




### Entities tracking

*Alice, Bob, Carl, Dan, and Emma each have a coin. All are dimes except Carl's. Alice and Carl trade coins.*

# State tracking task: LEGO

**LEGO (Learning Equality and Group Operations)** (Zhang et al., 2022): with answer up to $m < L$

$$\underbrace{\boxed{x_1 = g_1(x_0)},\ \ldots,\ \boxed{x_L = g_L(x_{L-1})}}_{\text{predicate clauses}} \quad \underbrace{\boxed{x_0 = y_0},\ \ldots,\ \boxed{x_m = y_m}}_{\text{answer clauses}}$$

with action $g_i \in \mathcal{G}$, value $y_i \in \mathcal{Y}$, variable $x_i \in \mathcal{X}$.

**Goal:** correctly calculate the last value $y_L = g_L(g_{L-1}(\cdots g_1(y_0)))$.

$$x_0 \xrightarrow{g_1} x_1 \xrightarrow{g_2} x_2 \xrightarrow{g_3} \cdots \xrightarrow{g_L} x_L, \quad \text{starting with } x_0 = y_0.$$

# State tracking task: LEGO

**LEGO (Learning Equality and Group Operations)** (Zhang et al., 2022): with answer up to $m < L$

$$\underbrace{\boxed{x_1 = g_1(x_0)}, \; \ldots, \; \boxed{x_L = g_L(x_{L-1})}}_{\text{predicate clauses}} \quad \underbrace{\boxed{x_0 = y_0}, \; \ldots, \; \boxed{x_m = y_m}}_{\text{answer clauses}}$$

with <u>action</u> $g_i \in \mathcal{G}$, <u>value</u> $y_i \in \mathcal{Y}$, <u>variable</u> $x_i \in \mathcal{X}$.

**Goal:** correctly calculate the last value $y_L = g_L(g_{L-1}(\cdots g_1(y_0)))$.

$$x_0 \xrightarrow{g_1} x_1 \xrightarrow{g_2} x_2 \xrightarrow{g_3} \cdots \xrightarrow{g_L} x_L, \quad \text{starting with } x_0 = y_0.$$

**Example:** modulo sum

$$x_1 = x_0 \oplus_5 4, \; x_2 = x_1 \oplus_5 2, \quad x_0 = 0, \, x_1 = ?, \, x_2 = ?$$

where

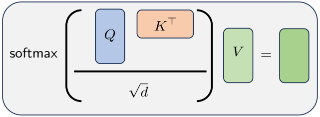
$$x_2 = x_1 \underbrace{\oplus_5 2}_{g_2} = (x_0 \underbrace{\oplus_5 4}_{g_1}) \oplus_5 2 = (\underbrace{\;0\;}_{y_0} \oplus_5 4) \oplus_5 2 \equiv 1 \pmod 5.$$

# One-layer attention + MLP model

**Model mechanism:** Attention (retrieval)

query

$$x_1 = g_1(x_0), \cdots \boxed{x_{m+1} = g_m(x_m)} \cdots, x_L = g_L(x_{L-1}), \quad x_0 = y_0, \cdots\cdots, \boxed{x_m = y_m}$$



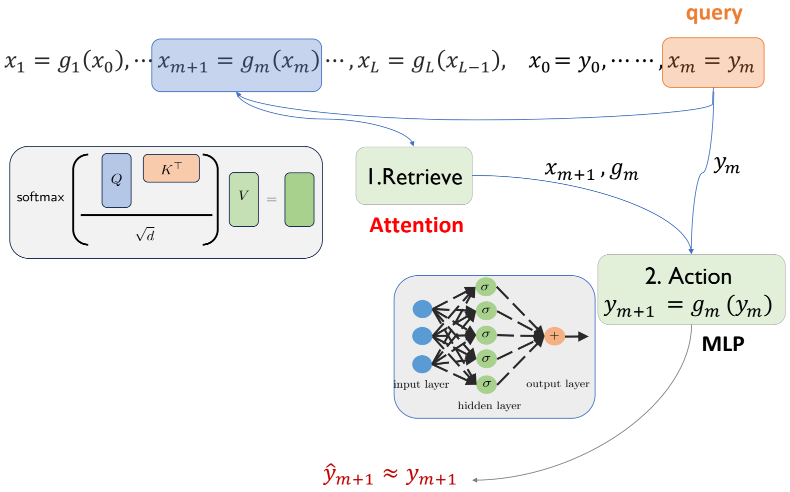

I.Retrieve

**Attention**

$$\hat{y}_{m+1} \approx y_{m+1}$$

# One-layer attention + MLP model

**Model mechanism:** FFN (action) $\circ$ Attention (retrieval)



# Tracking the attention dynamics

$$\mathbf{Z}^{L,m} \triangleq \underbrace{\boxed{x_1 = g_1(x_0)}}_{\mathbf{Z}_{\text{pred},1}}, \ \ldots, \ \boxed{x_L = g_L(x_{L-1})} \quad \underbrace{\boxed{x_0 = y_0}}_{\mathbf{Z}_{\text{ans},0}}, \ \ldots, \ \underbrace{\boxed{x_m = y_m}}_{\text{query } \mathbf{Z}_{\text{ans},m}}$$

**Training objective:** minimize the cross-entropy loss for *next-answer* prediction via gradient descent

$$\mathcal{L}(F) = \mathbb{E}\Big[\sum_{m=1}^{L} -\log p_F(y_{m+1} \mid \mathbf{Z}^{L,m})\Big]$$

where $d = |\mathcal{X}| + |\mathcal{G}| + |\mathcal{Y}|$ (roughly embedding dimension).

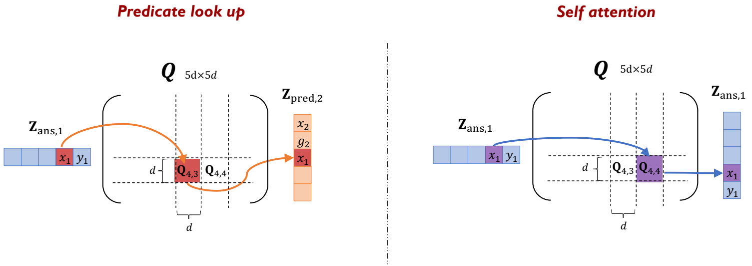
**Tracking** how attention concentrates on the key clauses and **quantify** the concentration *at loss convergence*

# Tracking the attention dynamics

$$\mathbf{Z}^{L,m} \triangleq \underbrace{\boxed{x_1 = g_1(x_0)}}_{\mathbf{Z}_{\text{pred},1}}, \ \ldots, \ \boxed{x_L = g_L(x_{L-1})} \quad \underbrace{\boxed{x_0 = y_0}}_{\mathbf{Z}_{\text{ans},0}}, \ \ldots, \ \underbrace{\boxed{x_m = y_m}}_{\text{query } \mathbf{Z}_{\text{ans},m}}$$

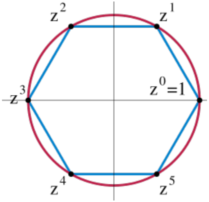
**Attention** $\mathbf{Attn}_{\text{ans},m\to\mathbf{k}}$: attention score from $\mathbf{Z}_{\text{ans},m} \to \mathbf{Z}_{\mathbf{k}}$;

**Uniform initialization:** attention scores are the same for all clauses.




Training dynamic: $\mathbf{Attn}_{\text{ans},\ell\to\text{pred},\ell} + \mathbf{Attn}_{\text{ans},\ell\to\text{ans},\ell} \to 1$

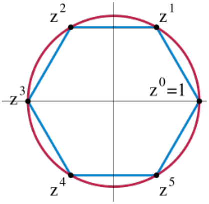
# Two group actions

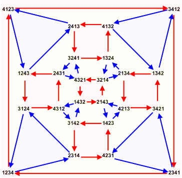



**Simply transitive action:**

- e.g., $C_n$, cyclic group (mod $n$)
- $\forall y, y'$, there is a **unique** $g \in \mathcal{G}$ s.t., $g(y) = y'$

# Two group actions



**Simply transitive action:**

- e.g., $C_n$, cyclic group (mod $n$)
- $\forall y, y'$, there is a **unique** $g \in \mathcal{G}$ s.t., $g(y) = y'$

**Symmetry action**:

- e.g., $S_n$, all permutations of $n$ elements
- $\mathcal{G}_{y \to y'} = \{g \in \mathcal{G} : \ g(y) = y'\}$ is **large**
- $S_n$ with $n \geq 5$ is in $\mathsf{NC}^1$.

# Length generalization for simple transitive group

**Theorem (Short-to-poly length generalization)**

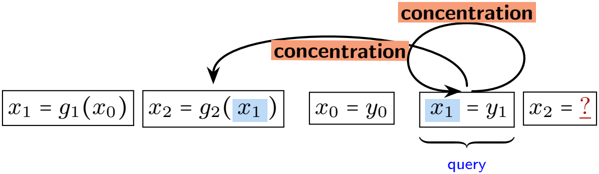
*For simply transitive $\mathcal{G}$, the model trained via GD on the LEGO task of $L \leq 2$ can directly solve tasks for $L \leq O(d^c)$ with some constant $0 < c < 1$.*

# Length generalization for simple transitive group

**Theorem (Short-to-poly length generalization)**

*For simply transitive $\mathcal{G}$, the model trained via GD on the LEGO task of $L \leq 2$ can directly solve tasks for $L \leq O(d^c)$ with some constant $0 < c < 1$.*

**Mechanism:** MLP perfectly captures action; while attention learns to concentrate on the **relevant** clauses.



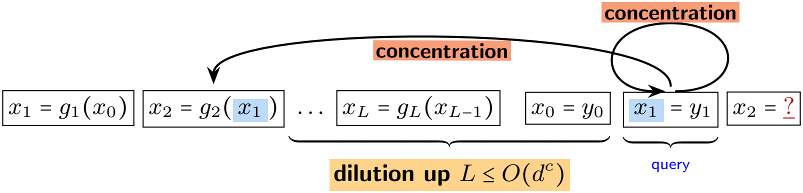

**Attention concentration**: $\mathbf{Attn}_{\mathsf{ans},\ell\to\mathsf{pred},\ell} + \mathbf{Attn}_{\mathsf{ans},\ell\to\mathsf{ans},\ell} = 1 - \frac{1}{O(d^c)}$

# Length generalization for simple transitive group

**Theorem (Short-to-poly length generalization)**

*For simply transitive $\mathcal{G}$, the model trained via GD on the LEGO task of $L \le 2$ can directly solve tasks for $L \le O(d^c)$ with some constant $0 < c < 1$.*

**Mechanism:** MLP perfectly captures action; while attention learns to concentrate on the **relevant** clauses.



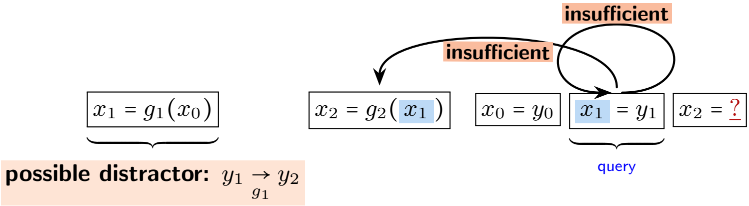

**Attention concentration**: $\mathbf{Attn}_{\text{ans},\ell\to\text{pred},\ell} + \mathbf{Attn}_{\text{ans},\ell\to\text{ans},\ell} = 1 - \frac{1}{O(d^c)}$

# Length generalization for symmetry group

**Hardness from symmetry** $|\mathcal{G}_{y\to y'}| \gg 1$, simply transitive $|\mathcal{G}_{y\to y'}| = 1$.

# Length generalization for symmetry group

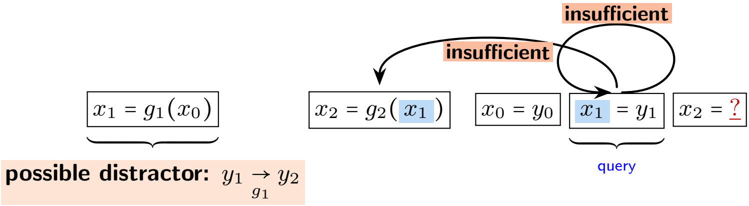
**Hardness from symmetry** $|\mathcal{G}_{y \to y'}| \gg 1$, simply transitive $|\mathcal{G}_{y \to y'}| = 1$.
Predicate lookup faces many **near-duplicates**:




$$\mathbf{Attn}_{\mathsf{ans},\ell \to \mathsf{pred},\ell} + \mathbf{Attn}_{\mathsf{ans},\ell \to \mathsf{ans},\ell} = 1 - \text{small constant}$$

# Length generalization for symmetry group

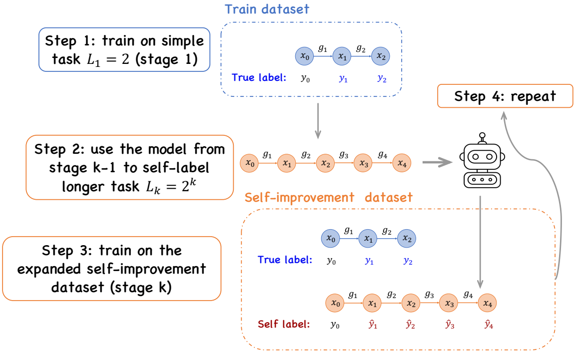
**Hardness from symmetry** $|\mathcal{G}_{y\to y'}| \gg 1$, simply transitive $|\mathcal{G}_{y\to y'}| = 1$.
Predicate lookup faces many **near-duplicates**:




$$\mathbf{Attn}_{\mathsf{ans},\ell\to\mathsf{pred},\ell} + \mathbf{Attn}_{\mathsf{ans},\ell\to\mathsf{ans},\ell} = 1 - \text{small constant}$$

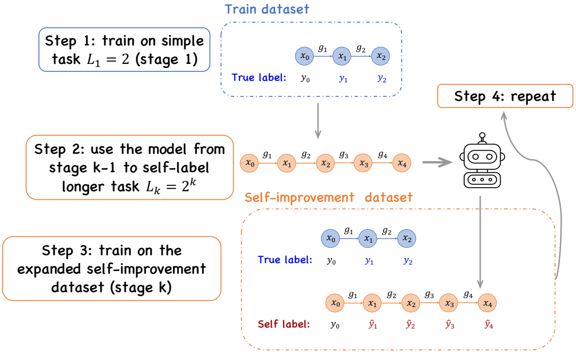
No guarantee beyond **constant**-factor length generalization!

# Self-improvement provably helps




Inspired by (Lee et al., 2025)

# Self-improvement provably helps

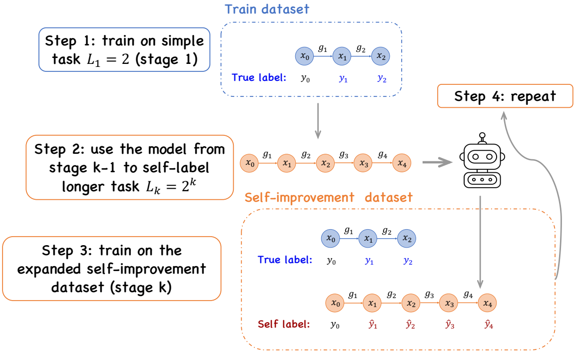



Inspired by (Lee et al., 2025)

**Theorem (Self-improvement for length generalization)**

*If $\mathcal{G}$ is symmetry $(S_n)$, then the model trained via self-training on $L = 1, 2, \cdots, 2^{k-1}$ can generalize to solve task of $L = 2^k$. After $\Theta(\log d)$ stages, it can solve tasks of $L \leq d$.*

# Self-improvement provably helps



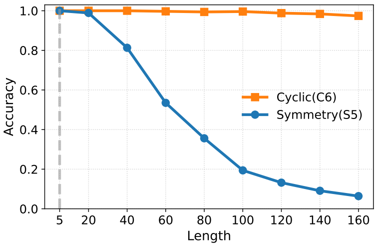

Inspired by (Lee et al., 2025)

**Theorem (Self-improvement for length generalization)**

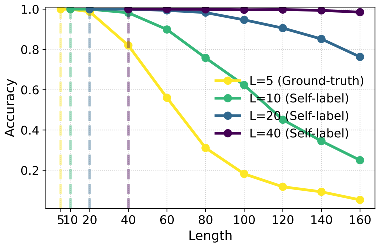
*If $\mathcal{G}$ is symmetry ($S_n$), then the model trained via self-training on $L = 1, 2, \cdots, 2^{k-1}$ can generalize to solve task of $L = 2^k$. After $\Theta(\log d)$ stages, it can solve tasks of $L \leq d$.*

Transformer with CoTs **learns** to solve problems in $\mathrm{NC}^1$!

# Numerical validation



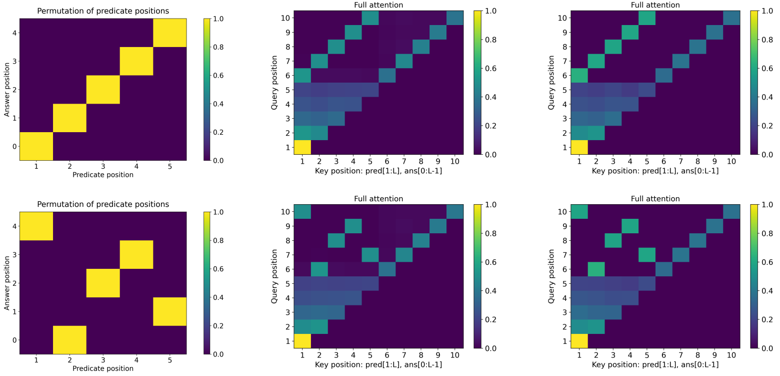

direct length generalization




self-improvement

Self-improvement improves length generalization for harder tasks

# Learned attention patterns

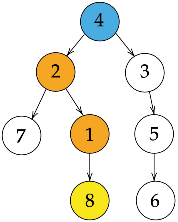



Ground truth $C_6$ $S_5$

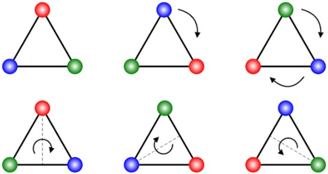
The learned attention patterns are adaptive to clause positions.

# Summary

**symbolic reasoning**




**reasoning with group action**



**Take-away:** one-layer transformers with CoT training can reason with length generalization (without positional encoding)

**Future directions:**

- Understanding RL for CoT reasoning.

# Thanks!

- Multi-head Transformers Provably Learn Symbolic Multi-step Reasoning via Gradient Descent, NeurIPS 2025, arXiv 2508.08222.
- Transformers Learn Chain-of-Thought Reasoning with Length Generalization, NeurIPS 2025, arXiv 2511.07378.

NSF

https://yuejiechi.github.io